

Name- Aaryan Pratap Singh

Sap id- 590032842

Batch - 15

1. Algorithm

Step 1: Start.

Step 2: Read the order n of the square matrix.

Step 3: Validate that $n > 0$.

Step 4: Declare a variable-length integer array $a[n][n]$.

Step 5: Read all matrix elements.

Step 6: Repeatedly display the menu:

1. Check whether all main diagonal elements are distinct.
2. Find the sum of main diagonal elements.
3. Display elements using diagonal traversal.
4. Exit.

Step 7: Use `switch` to call a separate function for the selected operation.

Step 8: Continue until the user selects Exit.

Step 9: Stop.

Function 1 — Distinct Main Diagonal Elements

1. Compare every main diagonal element $a[i][i]$ with every diagonal element after it.
2. If any two are equal, the diagonal elements are not distinct.
3. Otherwise, all main diagonal elements are distinct.

Function 2 — Sum of Main Diagonal

1. Initialize `sum = 0`.
2. For $i = 0$ to $n-1$, add $a[i][i]$ to `sum`.
3. Display `sum`.

Function 3 — Diagonal Traversal

For each diagonal having constant `row + column`:

- For even-numbered diagonals, traverse **upward-right**.
- For odd-numbered diagonals, traverse **downward-left**.
- Continue until all $n \times n$ elements have been displayed.

For the matrix

```
1 2 3
4 5 6
7 8 9
```

the traversal is: 1 2 4 7 5 3 6 8 9

2. Test Case Table

Test Case	Input Matrix	Operation	Expected Output
1	1 2 3 / 4 5 6 / 7 8 9	Distinct diagonal	1 5 9 are distinct
2	1 2 3 / 4 5 6 / 7 8 9	Diagonal sum	15
3	1 2 3 / 4 5 6 / 7 8 9	Traversal	1 2 4 7 5 3 6 8 9
4	1 2 / 3 1	Distinct diagonal	1 1 are not distinct
5		5 All operations	Distinct, sum = 5, traversal = 5
6	n = 0	Matrix size	Invalid; ask again

3. C Program

```
#include <stdio.h>

/* Function prototypes */

void checkDistinct(int n, int a[n][n]);

void diagonalSum(int n, int a[n][n]);

void diagonalTraversal(int n, int a[n][n]);

int main()

{

    int n, i, j, choice;

    /* Read and validate matrix order */

    do

    {

        printf("Enter the order of the square matrix: ");
```

```

scanf("%d", &n);

if (n <= 0)

    printf("Invalid input. Order must be positive.\n");

} while (n <= 0);

/* Variable-Length 2D Array */

int a[n][n];

printf("Enter the matrix elements:\n");

for (i = 0; i < n; i++)

{
    for (j = 0; j < n; j++)

    {

        scanf("%d", &a[i][j]);

    }

}

do

{

    printf("\n----- MENU ----- \n");

    printf("1. Check whether main diagonal elements are distinct\n");

    printf("2. Find sum of main diagonal elements\n");

    printf("3. Diagonal traversal\n");

    printf("4. Exit\n");

    printf("Enter your choice: ");

    scanf("%d", &choice);

    switch (choice)

    {

        case 1:

```

```

        checkDistinct(n, a);

        break;

    case 2:

        diagonalSum(n, a);

        break;

    case 3:

        diagonalTraversal(n, a);

        break;

    case 4:

        printf("Exiting program.\n");

        break;

    default:

        printf("Invalid choice. Please enter 1 to 4.\n");

    }

} while (choice != 4);

return 0;

}

/* Check whether all main diagonal elements are distinct */

void checkDistinct(int n, int a[n][n])

{

    int i, j;

    int distinct = 1;

    printf("Main diagonal elements: ");

    for (i = 0; i < n; i++)

    {

```

```

    printf("%d ", a[i][i]);

}

for (i = 0; i < n - 1; i++)

{

    for (j = i + 1; j < n; j++)

    {

        if (a[i][i] == a[j][j])

        {

            distinct = 0;

            break;

        }

    }

    if (distinct == 0)

        break;

}

if (distinct)

    printf("\nAll the main diagonal elements are distinct.\n");

else

    printf("\nAll the main diagonal elements are not distinct.\n");

}

/* Find sum of main diagonal elements */

void diagonalSum(int n, int a[n][n])

{

    int i;

    int sum = 0;

```

```

for (i = 0; i < n; i++)

{

    sum = sum + a[i][i];

}

printf("Sum of main diagonal elements: %d\n", sum);

}

/* Display matrix using diagonal zigzag traversal */

void diagonalTraversal(int n, int a[n][n])

{

    int d, row, col;

    printf("Diagonal traversal: ");

    /* d represents row + column */

    for (d = 0; d <= 2 * (n - 1); d++)

    {

        if (d % 2 == 0)

        {

            /*

            * Even diagonal:

            * Traverse downward-left

            */

            if (d < n)

            {

                row = 0;

                col = d;

```

```

    }

    else

    {

        row = d - n + 1;

        col = n - 1;

    }

    while (row < n && col >= 0)

    {

        printf("%d ", a[row][col]);

        row++;

        col--;

    }

}

else

{

    /*

    * Odd diagonal:

    * Traverse upward-right

    */

    if (d < n)

    {

        row = d;

        col = 0;

    }

    else

```

```

{

    row = n - 1;

    col = d - n + 1;

}

while (row >= 0 && col < n)

{

    printf("%d ", a[row][col]);

    row--;

    col++;

}

}

printf("\n");

}

```

Execution

```

[aaryanrajput@aaryans-macbook ~ % ./day35.c
Enter the order of the square matrix: 3
Enter the matrix elements:

1 2 3 4 5 6 7 8 9

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 5 9
All the main diagonal elements are distinct.

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements: 15

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 3
Diagonal traversal: 1 4 2 3 5 7 8 6 9

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 4
Exiting program.
aaryanrajput@aaryans-macbook ~ % █

```